

DEEP LEARNING FOR FINANCE

MODULE 4 | LESSON 2

Implementing univariate Recurrent Neural Networks (RNN)

Reading Time	40 minutes
---------------------	------------

Prior Knowledge	RNN, Market timing, Deep Neural Networks, Dropout
------------------------	---

Keywords	RNN, Market timing
-----------------	--------------------

1. Introduction and data gathering

In our first practical contact with Recurrent Neural Networks (RNNs) we will implement a model to predict stock **prices** (rather than returns, as we have seen in previous modules). This has an importance, because prices are closer to be interpreted as a sequences than returns. This makes them potentially more predictive using RNNs than returns.

As usual, we will start by downloading price data from Yahoo! finance. Note that we take prices from just one stock, our usual suspect Apple Inc., to explore the time-series univariate prediction case. In lesson 4 we will extend this to a multivariate case.

```
In [1]: import numpy as np
import pandas as pd
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
```

We start by gathering the data and defining the output as next day's close price:

```
In [2]: # Read the CSV file
file_path = 'AAPL.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=['Date'], da
df = df.sort_values('Date')
df = df.set_index('Date')

df['Output'] = df.Price.shift(-1)
# Reset the index to make Date a regular column
df = df.reset_index(names=['Date'])
# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()

df.head()
```

Out[2]:

	Date	Price	CVol	Change	% Change	% Return	Total Return (Gross)	Cumulative Return %	Open
2773	1991-12-02	0.46	118899320.0	0.01	1.97	1.97	0.48	276,80	0.45
2774	1991-12-03	0.45	102860910.0	-0.01	-2.42	-2.42	0.47	267,70	0.46
2776	1991-12-05	0.45	99198490.0	0.00	-0.99	-0.99	0.47	264,06	0.45
2777	1991-12-06	0.44	196985780.0	-0.01	-2.50	-2.50	0.46	254,96	0.44
2778	1991-12-09	0.44	97832080.0	0.00	0.77	0.77	0.46	257,69	0.44

In [3]:

```
# Convert 'Date' column to datetime for proper filtering
df['Date'] = pd.to_datetime(df['Date'])

# Filter for dates from 2010 onwards
df = df[df['Date'] >= '2010-01-01'].reset_index(drop=True)

print("Length of data frame: ", len(df))
df.head()
```

Length of data frame: 3852

Out[3]:

	Date	Price	CVol	Change	% Change	% Return	Total Return (Gross)	Cumulative Return %	Open	High
0	2010-01-04	7.64	493728720.0	0.12	1.56	1.56	8.37	6.427,19	7.62	7.64
1	2010-01-05	7.66	601904560.0	0.01	0.17	0.17	8.39	6.438,47	7.66	7.66
2	2010-01-06	7.53	552158800.0	-0.12	-1.59	-1.59	8.26	6.334,47	7.66	7.66
3	2010-01-07	7.52	477129720.0	-0.01	-0.18	-0.18	8.24	6.322,57	7.56	7.56
4	2010-01-08	7.57	447876700.0	0.05	0.66	0.66	8.29	6.365,27	7.51	7.57

2. Validation, train and test sample definition

Next, we proceed to define our training, test, and validation sets. Note that we will train using the 98% of the sample. The validation set will be 20% of the training test. We also need to define a 'window' of observations that we are going to take as a sequence input to our network. In this case we choose a window of 20, which means that the last 20 prices (observations) will be incorporated into the sequence of inputs for the network.

Note that not all of these may end up playing a big role in the prediction (this is for the training to decide), but we will give the network the possibility to consider up to 20 past prices.

Also, note that these, and others, parameters can be included as hyperparameters of the models that we can tune to achieve maximum performance of the algorithm. This is something we leave for future implementations of the Neural Network.

```
In [4]: # First, we deal with the definition of the training, test, and validations sets
val_split = 0.2
train_split = 0.98
train_size = int(len(df) * train_split)
val_size = int(train_size * val_split)
test_size = int(len(df) - train_size)

window_size = 20 # How many daily prices do I want to incorporate in the input

ts = test_size
split_time = len(df) - ts
test_time = df.iloc[split_time + window_size :, 0:1].values

# Second, we actually define the input matrix for train (X_train) and test (X_test)
# Note that these will need further work afterwards, in order to incorporate the
Xdf, ydf = df.iloc[:, 1:2], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

y_train_set = y[:split_time]
y_test_set = y[split_time:]

X_train_set = X[:split_time]
X_test_set = X[split_time:]

n_features = X_train_set.shape[1]

# Third, we proceed with scaling inputs to the model. Note how this is specially
scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train_set)
X_train_set_scaled = scaler_input.transform(X_train_set)
X_test_set_scaled = scaler_input.transform(X_test_set)

mean_ret = np.mean(y_train_set)

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
scaler_output.fit(y_train_set)
y_train_set_scaled = scaler_output.transform(y_train_set)

# Lastly, because we want a time series with up to 20 (window_size) past observations
training_time = df.iloc[:split_time, 0:1].values

X_train = []
y_train = []

for i in range(window_size, y_train_set_scaled.shape[0]):
    X_train.append(X_train_set_scaled[i - window_size : i, :])
```

```

y_train.append(y_train_set_scaled[i])

X_train, y_train = np.array(X_train), np.array(y_train)

print("Size of X vector in training:", X_train.shape)
print("Size of Y vector in training:", y_train.shape)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)
print("Size of X vector in test:", X_test.shape)
print("Size of Y vector in test:", y_test.shape)
print("Number of features in the model: ", n_features)

```

```

Size of X vector in training: (3754, 20, 1)
Size of Y vector in training: (3754, 1)
Size of X vector in test: (58, 20, 1)
Size of Y vector in test: (78, 1)
Number of features in the model: 1

```

3. Defining the RNN model

Once we have pre-processed the input and output data of our network, let's build it!

We will start with a very simple RNN that considers 5 hidden layers (4 simple RNN layers + 1 final dense layer). Each of the RNN layers will have 50 neurons. We apply a Dropout of 20% after each of the RNN layers. Finally, before the single-unit final output layer, we will apply a dense layer with 10 neurons. As you know already, this architecture is kind of random. We will play around with the composition of the network once we move to more advanced models through hyperparameter tuning.

A novel thing of RNNs is that we can set the 'return_sequences' argument to True or False, depending on whether we want to pass the last output (if False) or the whole sequence (if True) to the next layer. More info on this here:

https://www.tensorflow.org/api_docs/python/tf/keras/layers/SimpleRNN

```

In [5]: import tensorflow as tf

model = tf.keras.models.Sequential(
    [
        tf.keras.layers.SimpleRNN(
            50, return_sequences=True, input_shape=(X_train.shape[1], n_features
        ),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=True),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=True),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=False),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(10),
    ]
)

```

```

        tf.keras.layers.Dense(1),
    ]
)

```

2025-05-14 13:05:07.100468: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-05-14 13:05:07.148245: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-05-14 13:05:08.476403: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

/usr/local/lib/python3.11/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```

    super().__init__(**kwargs)

```

So, how does the model architecture look like?

In [6]: `model.summary()`

Model: "sequential"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 50)	2,600
dropout (Dropout)	(None, 20, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 50)	5,050
dropout_1 (Dropout)	(None, 20, 50)	0
simple_rnn_2 (SimpleRNN)	(None, 20, 50)	5,050
dropout_2 (Dropout)	(None, 20, 50)	0
simple_rnn_3 (SimpleRNN)	(None, 50)	5,050
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 10)	510
dense_1 (Dense)	(None, 1)	11

Total params: 18,271 (71.37 KB)

Trainable params: 18,271 (71.37 KB)

Non-trainable params: 0 (0.00 B)

Note how here the number of parameters to be trained by the model is defined a little bit different at each step. For example, for the second RNN layer, note that we have a total of 5,050 parameters. Where does this number come from?

Well, $50 \times 50 + 50 \times 50 + 50 \times 1 = 5,050$ parameters. In other words, outputs from previous layer (50) times neurons in this layer (50) plus the neurons in the layer times themselves (50×50), plus the bias term vector (50×1).

Where does the 2,600 parameters of the first layer come from? Well, 50×50 (neurons in the layer) + 50×1 (neurons times number of model features) + 50×1 (bias terms) = 2,600

Alike we did for other models, if we sum up all the parameters in the different layers we end up with 18,271 parameters to be trained in this basic model.

4. Model training

Now, we can proceed with model training. As you will see, there is nothing really new about this and basically training works in exactly the same way once we have defined our hidden layers as 'recurrent' ones.

```
In [7]: from keras.callbacks import EarlyStopping

hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr), loss="mean_absolute
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=1, patience=10, restore_best_weights
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=500,
    batch_size=64,
    verbose=1,
    callbacks=[es],
)
```

Epoch 1/500			
47/47		5s 27ms/step	loss: 0.8755 - val_loss: 0.5018
Epoch 2/500			
47/47		2s 15ms/step	loss: 0.7682 - val_loss: 0.3504
Epoch 3/500			
47/47		1s 15ms/step	loss: 0.6676 - val_loss: 0.2727
Epoch 4/500			
47/47		1s 17ms/step	loss: 0.6242 - val_loss: 0.2808
Epoch 5/500			
47/47		1s 15ms/step	loss: 0.5754 - val_loss: 0.1098
Epoch 6/500			
47/47		1s 15ms/step	loss: 0.5630 - val_loss: 0.1120
Epoch 7/500			
47/47		1s 15ms/step	loss: 0.5124 - val_loss: 0.1234
Epoch 8/500			
47/47		1s 15ms/step	loss: 0.4827 - val_loss: 0.1160
Epoch 9/500			
47/47		1s 15ms/step	loss: 0.4461 - val_loss: 0.0858
Epoch 10/500			
47/47		1s 15ms/step	loss: 0.4200 - val_loss: 0.1336
Epoch 11/500			
47/47		1s 15ms/step	loss: 0.3870 - val_loss: 0.0733
Epoch 12/500			
47/47		1s 15ms/step	loss: 0.3677 - val_loss: 0.0799
Epoch 13/500			
47/47		1s 15ms/step	loss: 0.3573 - val_loss: 0.0771
Epoch 14/500			
47/47		1s 16ms/step	loss: 0.3136 - val_loss: 0.1388
Epoch 15/500			
47/47		1s 15ms/step	loss: 0.3070 - val_loss: 0.0667
Epoch 16/500			
47/47		1s 15ms/step	loss: 0.2891 - val_loss: 0.2095
Epoch 17/500			
47/47		1s 15ms/step	loss: 0.2694 - val_loss: 0.0884
Epoch 18/500			
47/47		1s 15ms/step	loss: 0.2530 - val_loss: 0.0768
Epoch 19/500			
47/47		1s 15ms/step	loss: 0.2293 - val_loss: 0.1024
Epoch 20/500			
47/47		1s 15ms/step	loss: 0.2189 - val_loss: 0.0873
Epoch 21/500			
47/47		1s 15ms/step	loss: 0.1981 - val_loss: 0.0665
Epoch 22/500			
47/47		1s 15ms/step	loss: 0.1845 - val_loss: 0.0676
Epoch 23/500			
47/47		1s 15ms/step	loss: 0.1743 - val_loss: 0.0637
Epoch 24/500			
47/47		1s 15ms/step	loss: 0.1633 - val_loss: 0.0608
Epoch 25/500			
47/47		1s 15ms/step	loss: 0.1587 - val_loss: 0.0647
Epoch 26/500			
47/47		1s 15ms/step	loss: 0.1468 - val_loss: 0.0677
Epoch 27/500			
47/47		1s 15ms/step	loss: 0.1421 - val_loss: 0.0674
Epoch 28/500			
47/47		1s 15ms/step	loss: 0.1300 - val_loss: 0.0644
Epoch 29/500			
47/47		1s 15ms/step	loss: 0.1202 - val_loss: 0.0639
Epoch 30/500			
47/47		1s 15ms/step	loss: 0.1203 - val_loss: 0.0648

```
Epoch 31/500
47/47 ————— 1s 15ms/step - loss: 0.1103 - val_loss: 0.0630
Epoch 32/500
47/47 ————— 1s 15ms/step - loss: 0.1074 - val_loss: 0.0618
Epoch 33/500
47/47 ————— 1s 15ms/step - loss: 0.1037 - val_loss: 0.0761
Epoch 34/500
47/47 ————— 1s 16ms/step - loss: 0.0978 - val_loss: 0.0626
Epoch 34: early stopping
Restoring model weights from the end of the best epoch: 24.
```

```
Out[7]: <keras.src.callbacks.history.History at 0x714a7b4dfe90>
```

Next, as usual, we will visualize our predictions in the test sample, undergoing the usual inverse transform to un-do the scaling process:

```
In [8]: prediction = model.predict(X_test)
prediction = scaler_output.inverse_transform(prediction)
prediction = prediction.flatten()

values = np.array(y_test[window_size:])
values = values.flatten()
```

```
2/2 ————— 1s 310ms/step
```

```
In [9]: df_predictions = pd.DataFrame(
        {"Date": test_time.flatten(), "Pred RNN": prediction, "values": values}
    )
df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%Y-%m-%d")
df_predictions.head()
```

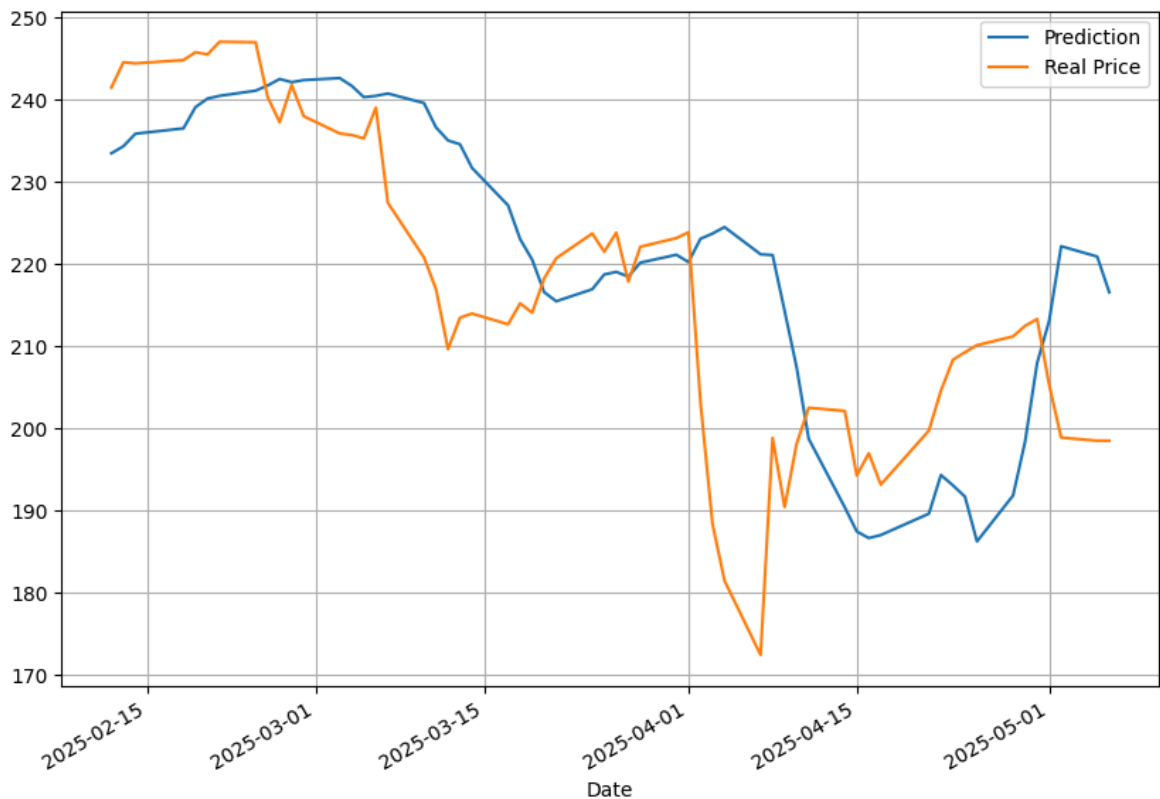
```
Out[9]:
```

	Date	Pred RNN	values
0	2025-02-12	233.512192	241.529999
1	2025-02-13	234.372055	244.600006
2	2025-02-14	235.897598	244.470001
3	2025-02-18	236.552780	244.869995
4	2025-02-19	239.142578	245.830002

And, finally, in a plot:

```
In [10]: import matplotlib.pyplot as plt

plt_1 = plt.figure(figsize=(10, 7))
ax = plt.gca()
df_predictions.plot(x="Date", y="Pred RNN", label="Prediction", ax=ax)
df_predictions.plot(x="Date", y="values", label="Real Price", ax=ax)
plt.grid()
plt.show()
```



As you can see, our model does not perform so well for predicting the time-series of prices given a sequence. It kind of 'guesses' part of the trend, but is far from being a nice tool to closely predict future stock prices.

5. Conclusion

In this lesson we have implemented our first Recurrent Neural Network (RNN) using a sequence of 20 past prices to predict future ones. We have seen that the predictive performance of this model is not so great after all. But there are lots of things to be done in terms of improving the model. This is what we will tackle in the next lessons, which will also serve as a good excuse to introduce the details of how forward and backprop work in RNN models.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.